APTURA TECH SOLUTIONS

Python Development Internship – Batch 02

Week 02 Task Report

Library Management System – Advanced Version

Submitted By:	Hafiz Siddiqui
Program:	BS Software Engineering, SMIU Karachi
Internship Track:	Python Development Department
Task:	Task 01 – Library Management System (Advanced Version)
Date:	July 21, 2026

1. Objective

Design and build an advanced, object-oriented **Library Management System** covering librarian authentication, book issuing/returning with automatic fine calculation, persistent relational storage, search & filter, structured exception handling, and a full audit-log trail – then reason at a system-design level about (1) making the application safe for multiple librarians working at once, and (2) which storage technology best fits the project and why.

2. System Architecture

The system is organised as a small, testable Python package rather than one long script, so each concern (auth, storage, business rules, presentation) can be reasoned about and changed independently – standard OOP/modular design practice.

Module	Responsibility
main.py	Entry point – launches the CLI.
library_system/exceptions.py	Custom exception hierarchy (LibraryError and subclasses) so the CLI can catch precisely what went wrong.
library_system/logger_setup.py	Central logging configuration – writes every action and error to library.log with a timestamp.
library_system/database.py	SQLite connection, schema creation, WAL mode, retry-on-lock helper, and an atomic transaction context manager.
library_system/auth.py	Librarian registration & login; PBKDF2-HMAC-SHA256 password hashing with a per-user salt.
library_system/services.py	Core business logic: books, members, issuing/returning, fine calculation, search & filter, reports.
library_system/cli.py	Menu-driven interface tying authentication and services together, with exception handling around every action.

2.1 Database schema (SQLite, library.db)

Four related tables, with foreign keys tying transactions to the specific book and member involved: `users` (librarian accounts), `books`, `members`, and `transactions` (issue/return records, including who processed each one via `processed_by` – important once multiple librarians are involved, see Question 01).

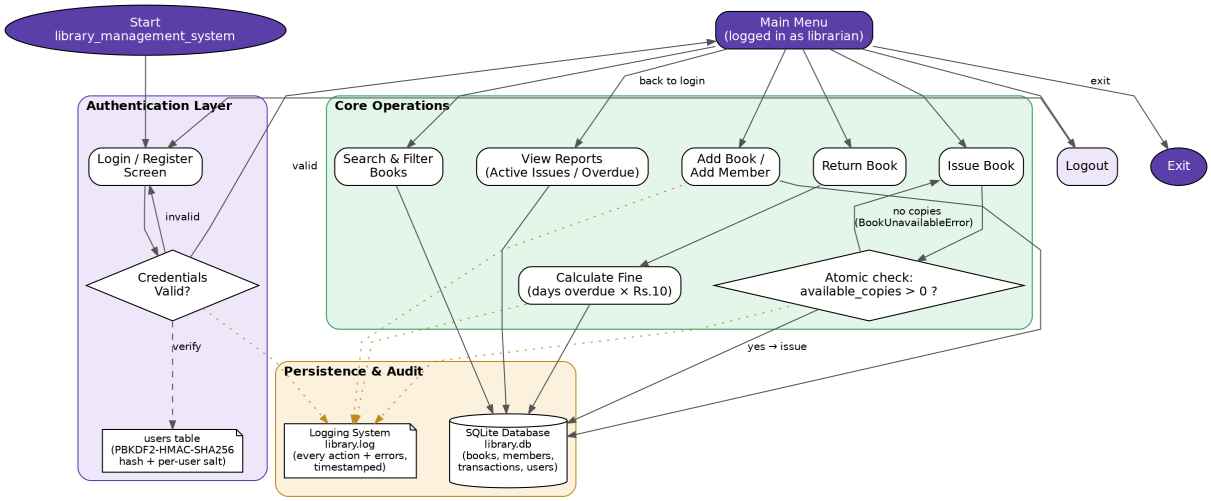
2.2 Features implemented

- **User Authentication** – librarian accounts with PBKDF2-HMAC-SHA256 password hashing (100,000 iterations) and a unique random salt per user; no plaintext passwords are ever stored.
- **Book Issue & Return** – a 14-day loan period; issuing uses a single atomic SQL statement to check-and-decrement available copies (closing the race window discussed in Question 01).
- **Fine Calculation** – Rs. 10 per day overdue, computed automatically on return; verified end-to-end with a dedicated demonstration (Section 6).
- **File/Database Storage** – a real relational SQLite database (library.db) with indexed primary keys and foreign-key relationships, not a flat file.

- **Search & Filter** – keyword search across title/author/category/ISBN, filter by availability, and filter by category, all as indexed SQL queries.
- **Exception Handling** – a custom exception hierarchy (BookNotFoundError, BookUnavailableError, DuplicateISBNError, MemberNotFoundError, AuthenticationError, TransactionError, ValidationError, …); every menu action is wrapped so a single bad input or business-rule violation never crashes the session.
- **Logging System** – every login, registration, book/member addition, issue, return, fine, and error is written to library.log with a timestamp, giving a full audit trail.

3. Project Flow Diagram

The diagram below traces a librarian's session from login through the main operations to the persistence and audit layer.



4. Question 01 – Supporting Multiple Simultaneous Librarians

As submitted, the application already avoids the most dangerous failure mode – two librarians both issuing the last copy of the same book – by using a single atomic SQL statement rather than a "read, check in Python, then write" sequence:

```
UPDATE books SET available_copies = available_copies - 1
WHERE book_id = ? AND available_copies > 0;
```

Because the check (`available_copies > 0`) and the write happen as one indivisible database operation, only one concurrent caller can ever win when copies run out – the database itself serializes the race, rather than the application trying (and risking failing) to do so.

4.1 Empirical verification

This wasn't just assumed – it was tested. A supplementary script (`bonus_concurrency_test.py`) spawns 10 separate OS processes, each with its own database connection, all attempting to issue the *same* book that has only 1 copy, at effectively the same instant:

Metric	Result	Expected
Concurrent librarians attempting to issue the last copy	10	10
Successful issues	1	1
Cleanly blocked (BookUnavailableError, no crash)	9	9
Unexpected errors	0	0
Final available_copies in the database	0	0 (never negative)

This result was consistent across repeated runs – no overselling ever occurred.

4.2 Where SQLite still falls short, and the full redesign

The atomic-guard pattern above protects data *correctness* even today, but SQLite is still fundamentally a single-file, single-active-writer engine: it is not designed to be opened for writing by many client machines over a network at once, and doing so risks file corruption rather than just slow performance. A production redesign for several librarians – potentially at different desks or branches – working simultaneously would involve:

- **Move to a client-server DBMS (MySQL or PostgreSQL).** Every librarian's client (desktop app, web app, or terminal) connects over the network to one centralized database server, instead of each opening its own local file. This is the single most important change – a shared server is what real multi-writer concurrency requires.
- **Keep the same atomic, guarded-UPDATE pattern – now backed by server-side row-level locking.** The exact statement above (`UPDATE ... WHERE available_copies > 0`) works unchanged on MySQL/PostgreSQL, and the server locks only the affected row, so librarians issuing *different* books never block each other – only concurrent attempts on the *same* book are serialized.
- **Keep a per-transaction audit trail.** The `processed_by` column already implemented records exactly which librarian handled each issue/return – essential once more than one person can make changes, both for accountability and for resolving disputes.
- **Session-token based authentication.** Replace one-shot username/password checks with a session token issued at login and validated on every request, so a networked client doesn't resend credentials constantly and an admin can revoke a specific librarian's session immediately if needed.

- **Connection pooling** at the application layer so many simultaneous librarian sessions share a small set of database connections efficiently, rather than each opening (and holding) its own.
- **Client-side refresh before confirming a risky action.** The server-side atomic guard already prevents any actual data corruption, but for good UX the client should re-check current availability immediately before confirming an issue, so a librarian isn't routinely surprised by a same-second conflict.

In short: the current implementation already applies the correct *algorithmic* pattern for safe concurrent updates (and proves it empirically); the redesign for real multi-librarian use is primarily an *infrastructure* change – moving from an embedded, single-file database to a networked, multi-client one that was built for exactly this purpose. That conclusion feeds directly into Question 02.

5. Question 02 – Storage Options Compared

JSON, CSV, SQLite, and MySQL were compared against three criteria: scalability (how it holds up as records and concurrent users grow), performance (query/lookup speed), and security (protection of the data and controlled access to it).

	Scalability	Performance	Security
JSON	Poor – whole file re-read/rewritten per change; no indexing; O(n) lookups.	Fine only for small datasets; parsing cost grows with file size.	None built-in; no access control; concurrent writes can corrupt the file.
CSV	Poor – flat, non-relational; joining related data (e.g. a transaction to a book and a member) must be done manually in application code.	Similar limits to JSON, plus fragile parsing (quoting/escaping issues).	None built-in; plaintext; no access control.
SQLite	Good for a single-machine app; WAL mode allows concurrent readers with one writer; not built for many networked writers at once.	Excellent for a single app – indexed SQL queries, no network round-trip.	Parameterized queries prevent SQL injection; no user-level DB accounts – security is enforced at the OS/file and application level.
MySQL	Excellent – built for many simultaneous client connections; scales further via replication and read replicas.	High concurrent throughput with proper indexing; connection pooling and caching layers can be added on top.	Built-in user accounts with GRANT/REVOKE privileges, SSL/TLS connections, and mature backup/audit tooling.

5.1 Choice for this deliverable: SQLite

For the system as submitted – a single-user-at-a-time console application meant to run standalone, with no server to install – SQLite is the right fit: zero configuration, ships with Python's standard library, and still provides real indexed SQL for search/filter and joined reports (see Sections 8–9), rather than the linear scans a JSON or CSV approach would force. It also let us demonstrate the atomic-transaction pattern from Question 01 using a real, indivisible database operation.

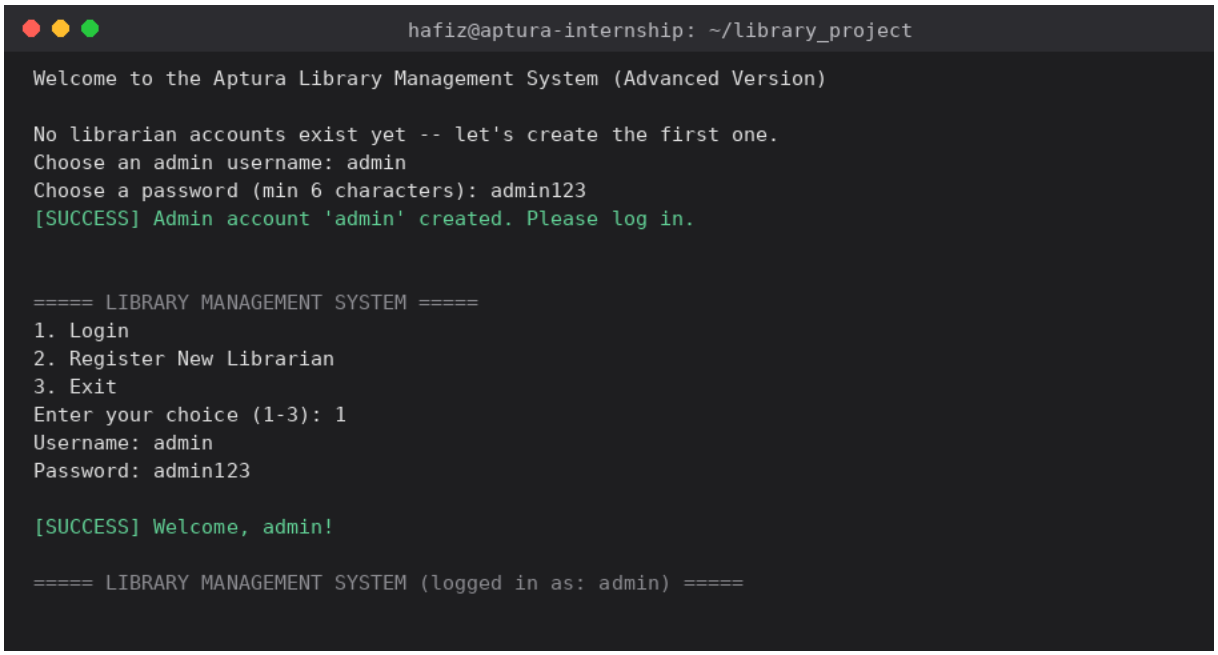
5.2 Choice for a production, multi-librarian redesign: MySQL

If the system is redesigned per Question 01 for several librarians – possibly at different desks or branches – working simultaneously, MySQL (or PostgreSQL) becomes the better choice, precisely because that is the scenario SQLite is not built for: many concurrent network clients writing at once, needing row-level locking, connection pooling, and per-user database accounts with restricted privileges. As the catalogue and transaction history grow over years of operation, MySQL's replication and read-replica support also give a clear path to scaling further.

JSON and CSV were ruled out for anything beyond the smallest, single-user, non-concurrent prototype – they have no indexing, no locking, and no relational integrity, and using them here would mean re-implementing (slowly, and error-prone) everything a real database already does well. This mirrors the same conclusion reached for the Week 01 Student Record system, now confirmed again for a more relational, multi-table project.

6. Program Execution & Output Screenshots

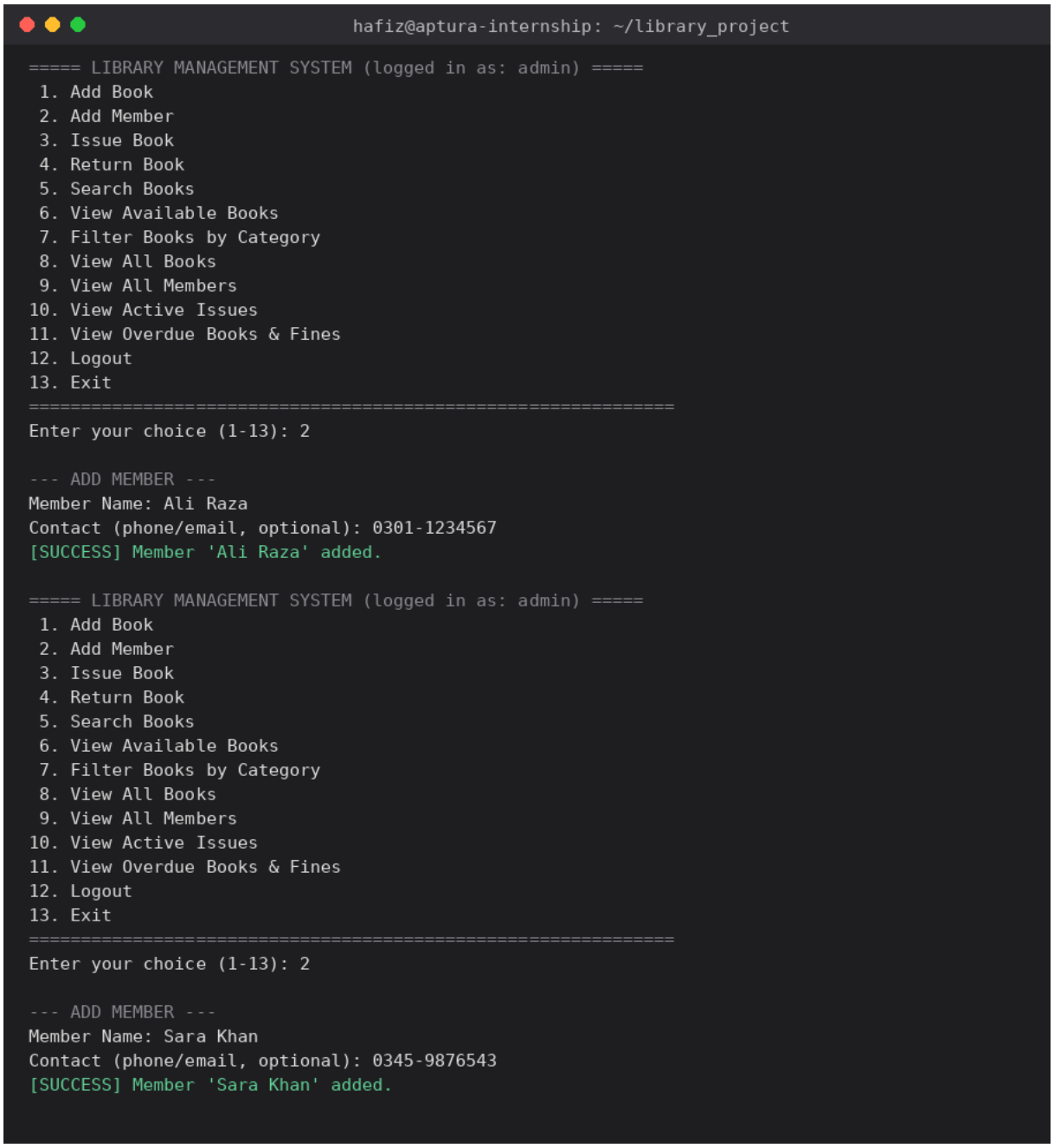
The system was run end-to-end in a live terminal session covering every menu option, including validation and error handling, followed by a dedicated demonstration of fine calculation on a genuinely overdue return.



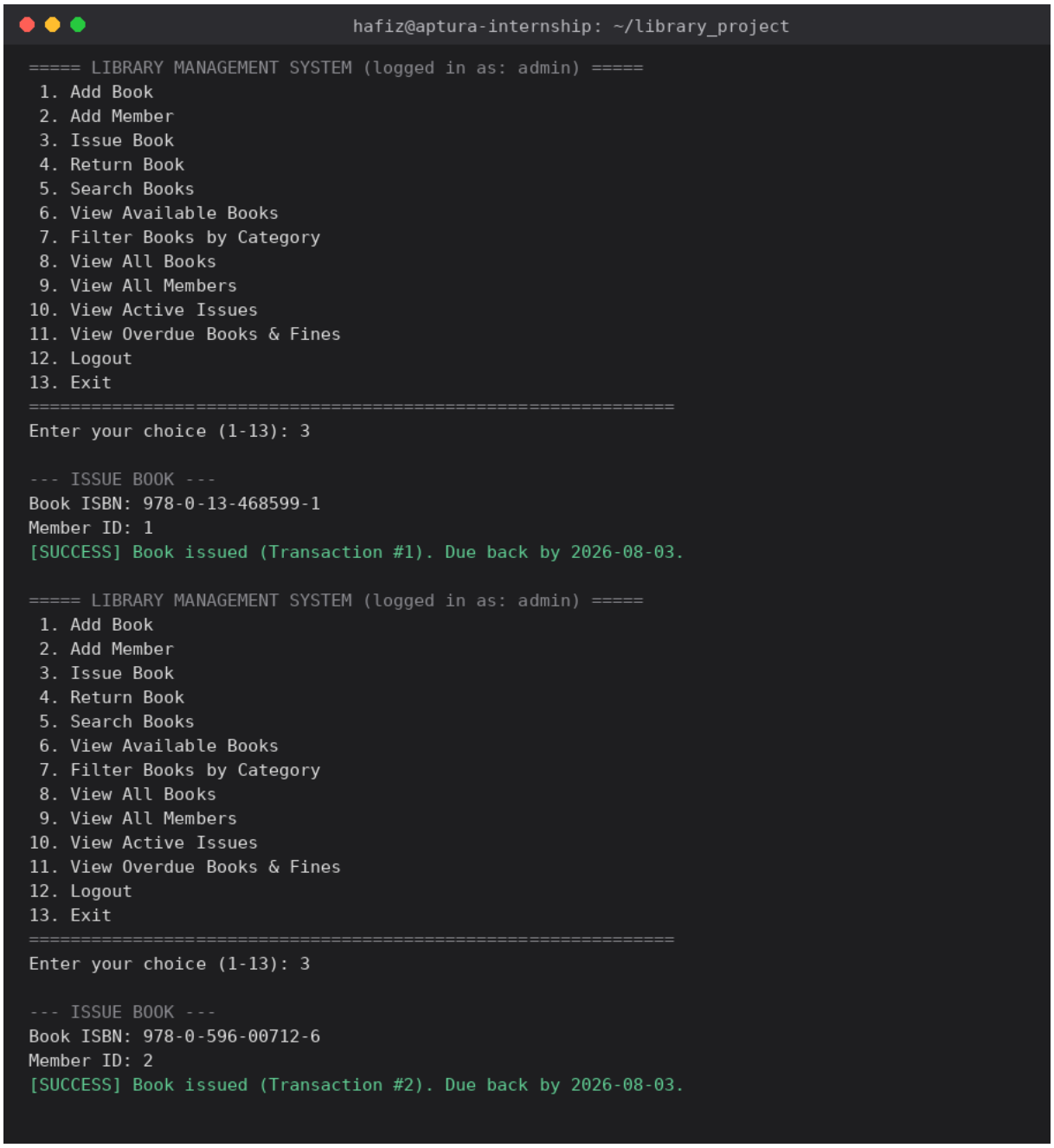
First run: no librarian accounts exist yet, so the system bootstraps the first admin account, then logs in.

Adding the first two books to the catalogue.

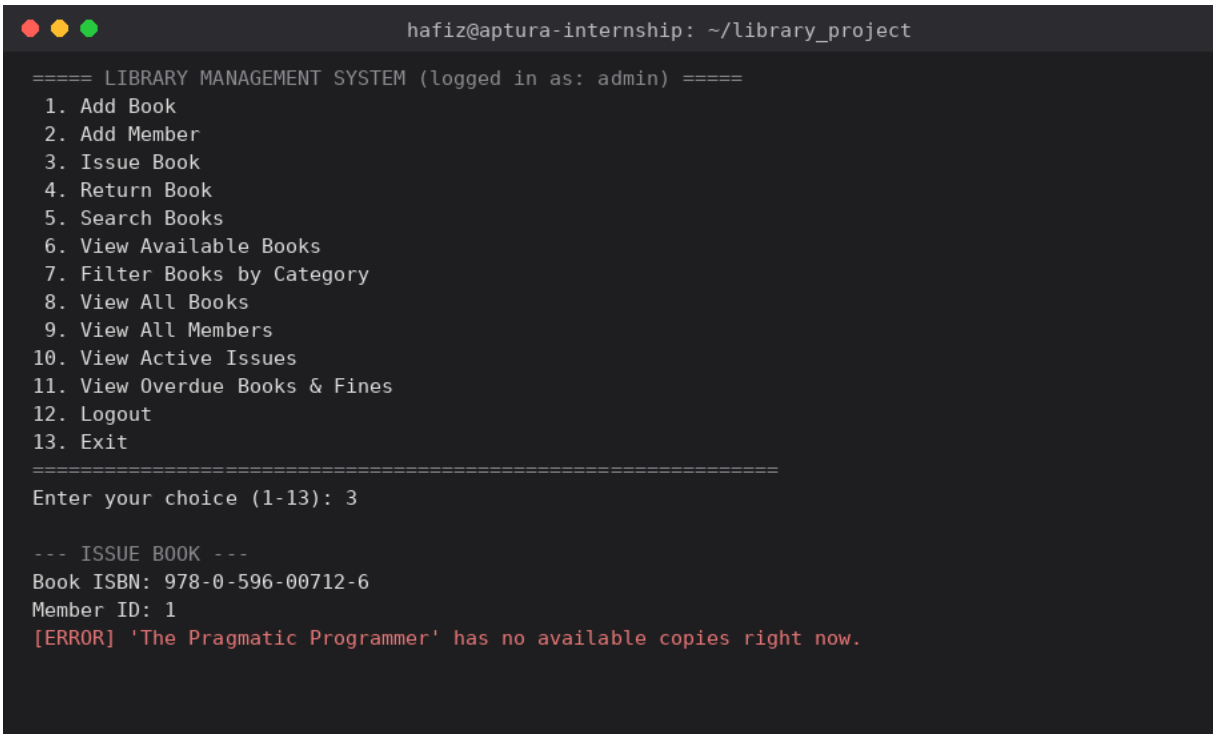
Adding the third book, then correctly rejecting a duplicate ISBN.



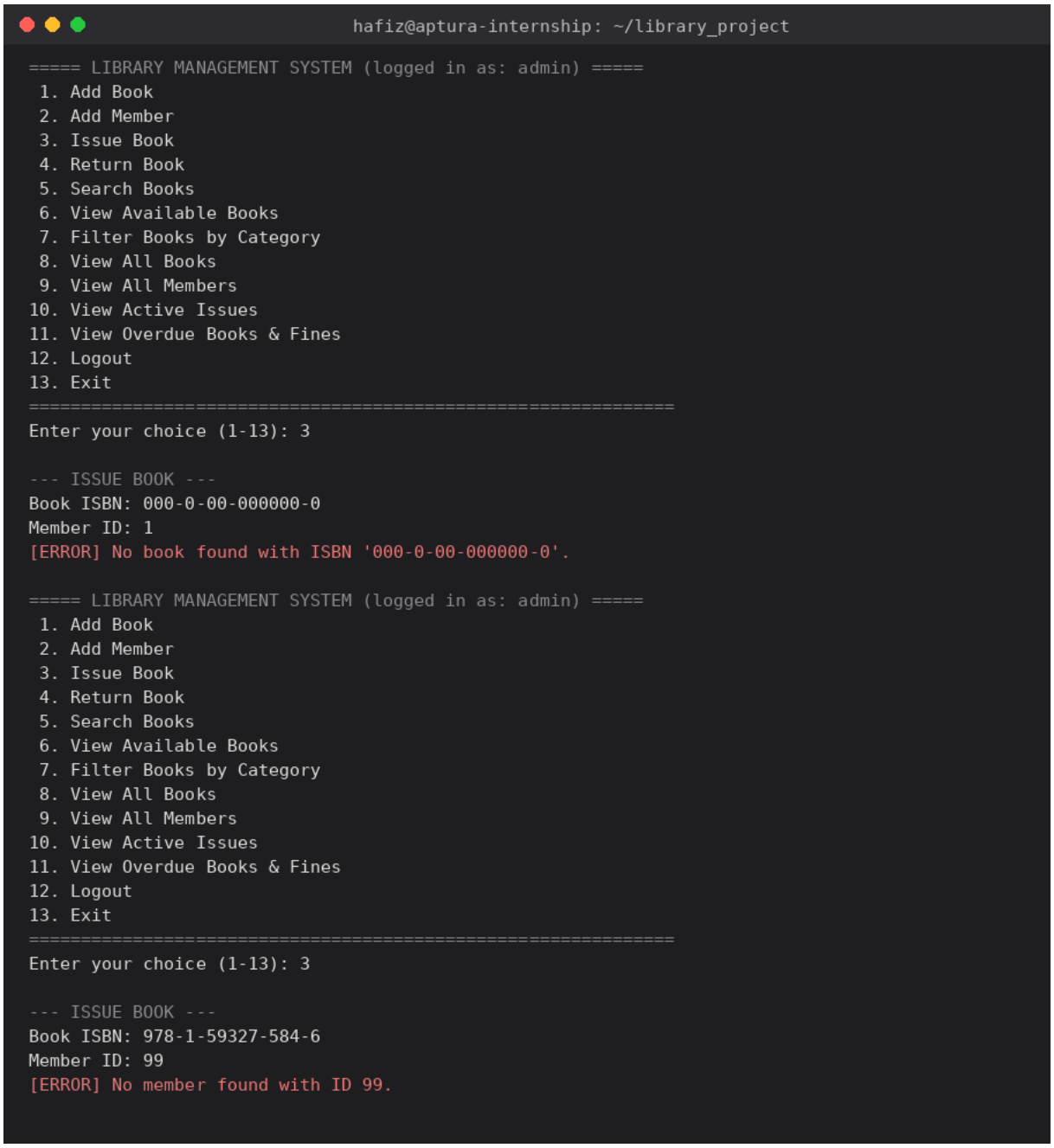
Adding two library members.



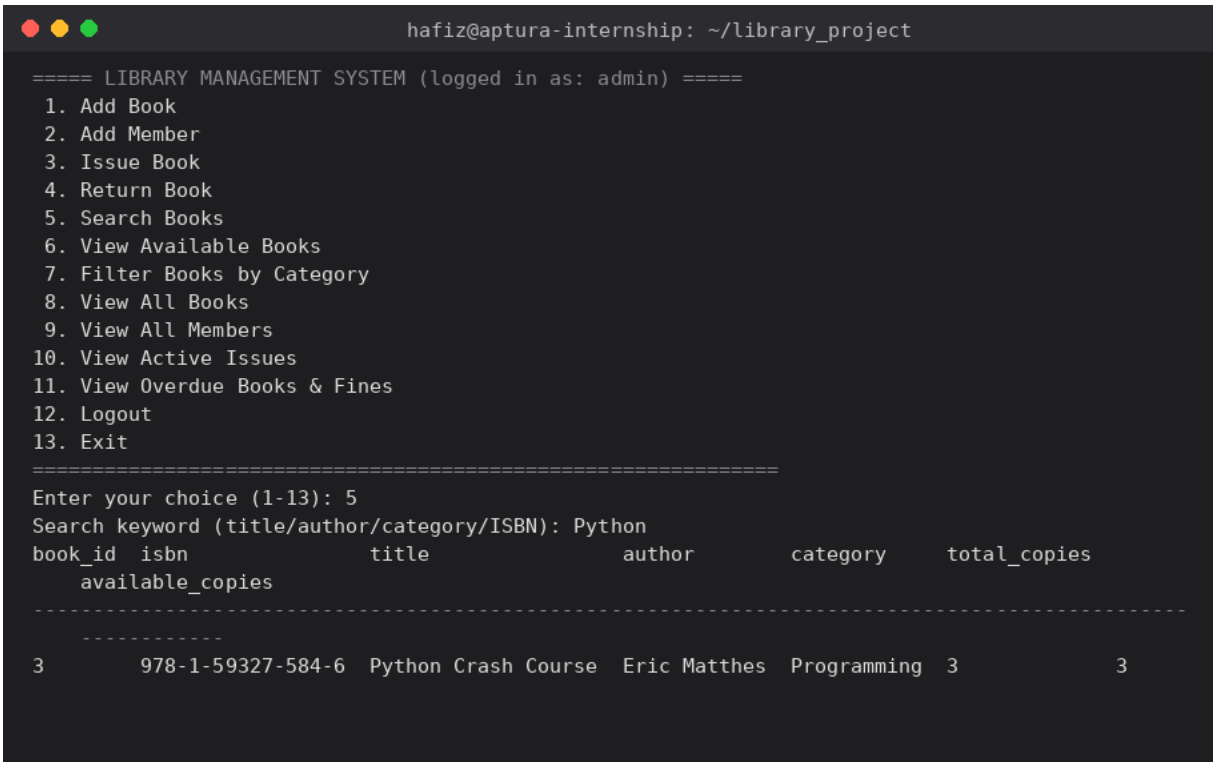
Issuing books to members – two successful issues, each with a computed due date (14-day loan period).



Attempting to issue a book with zero copies remaining – cleanly rejected with BookUnavailableError.

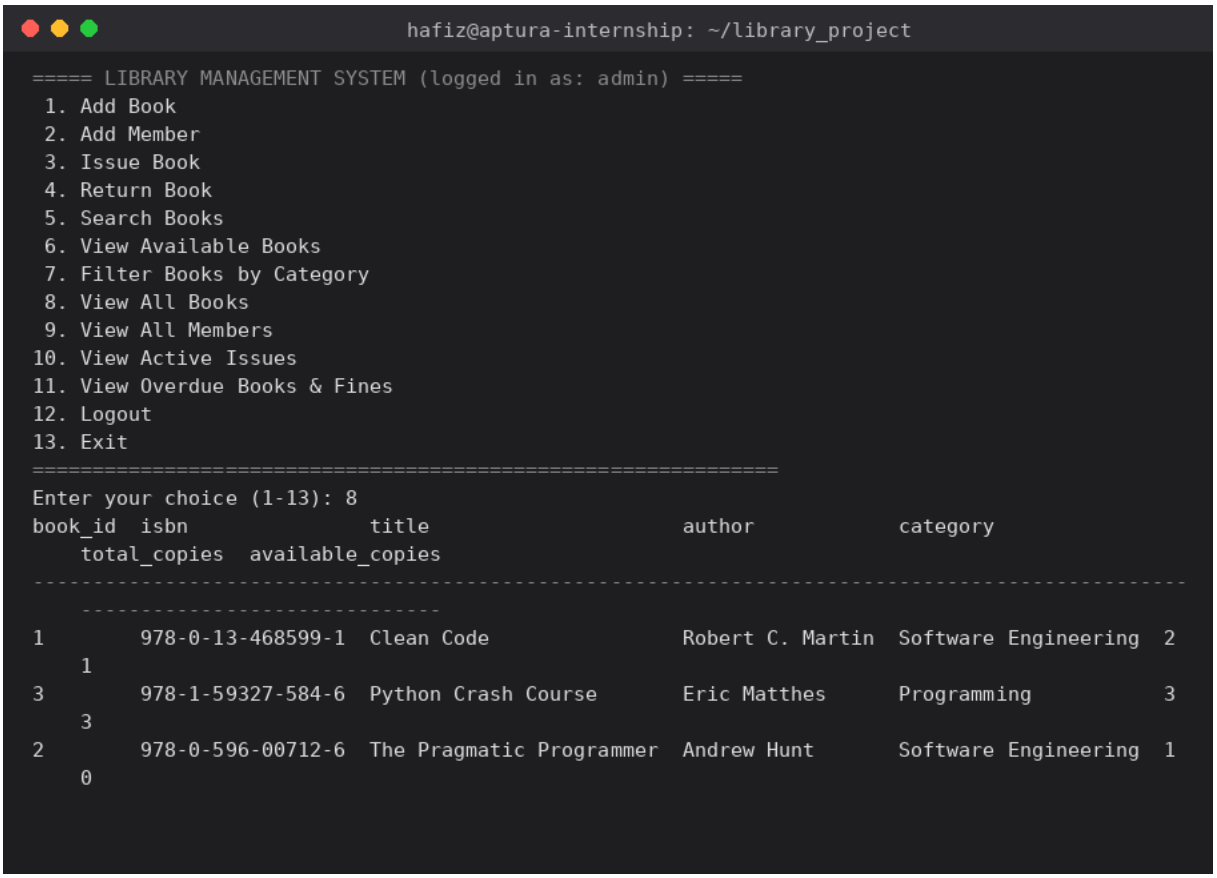


Attempting to issue a non-existent ISBN and a non-existent member ID – both cleanly rejected.

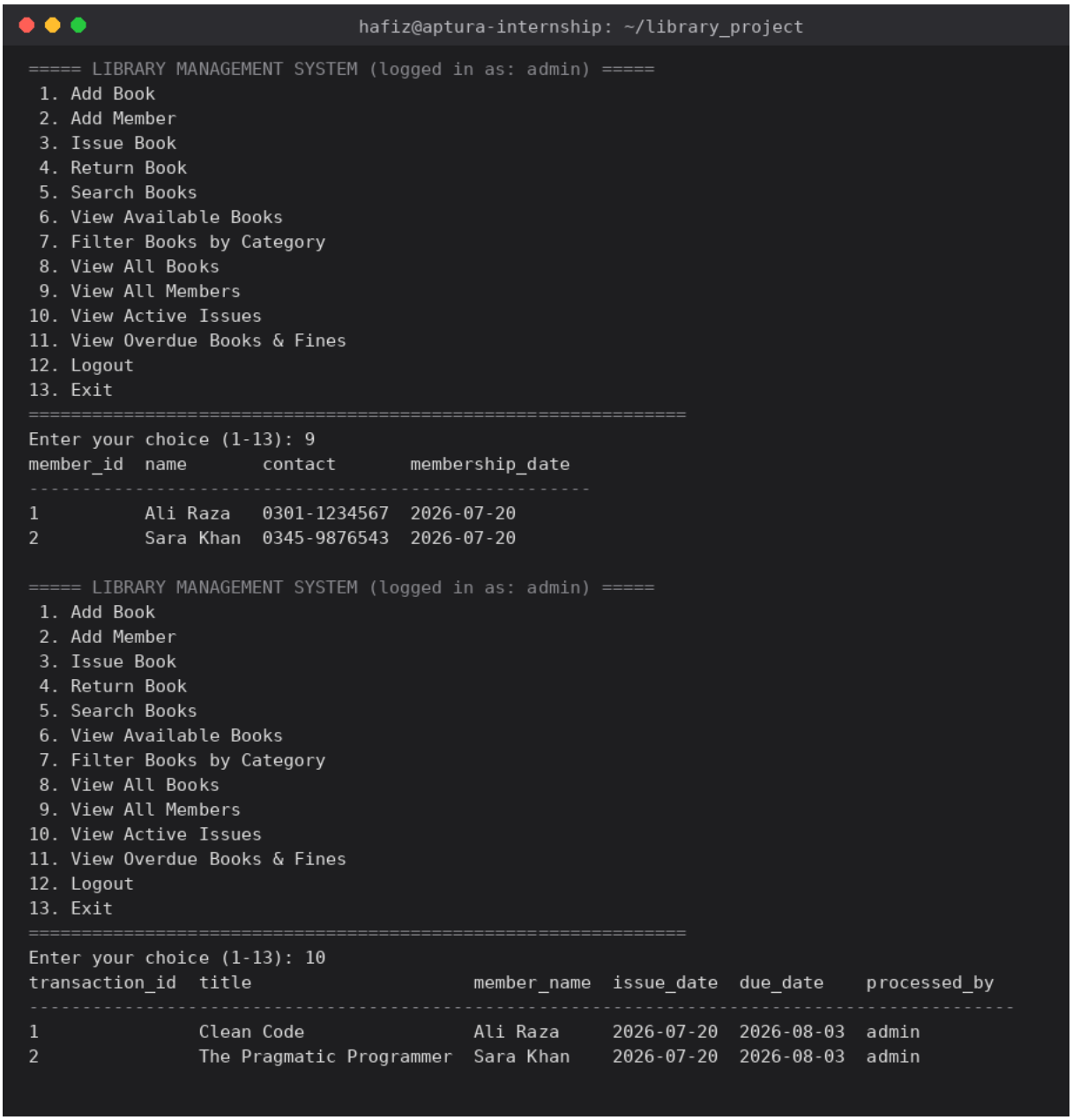


Keyword search across title/author/category/ISBN.

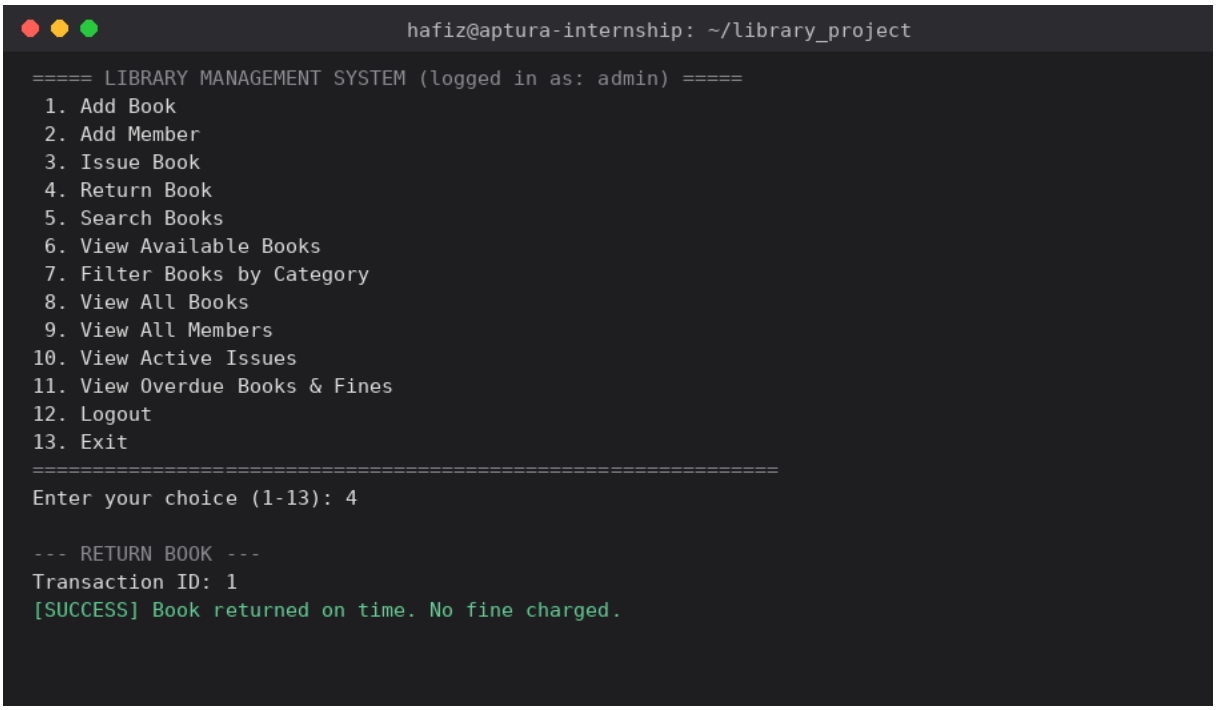
Filtering by availability and by category.



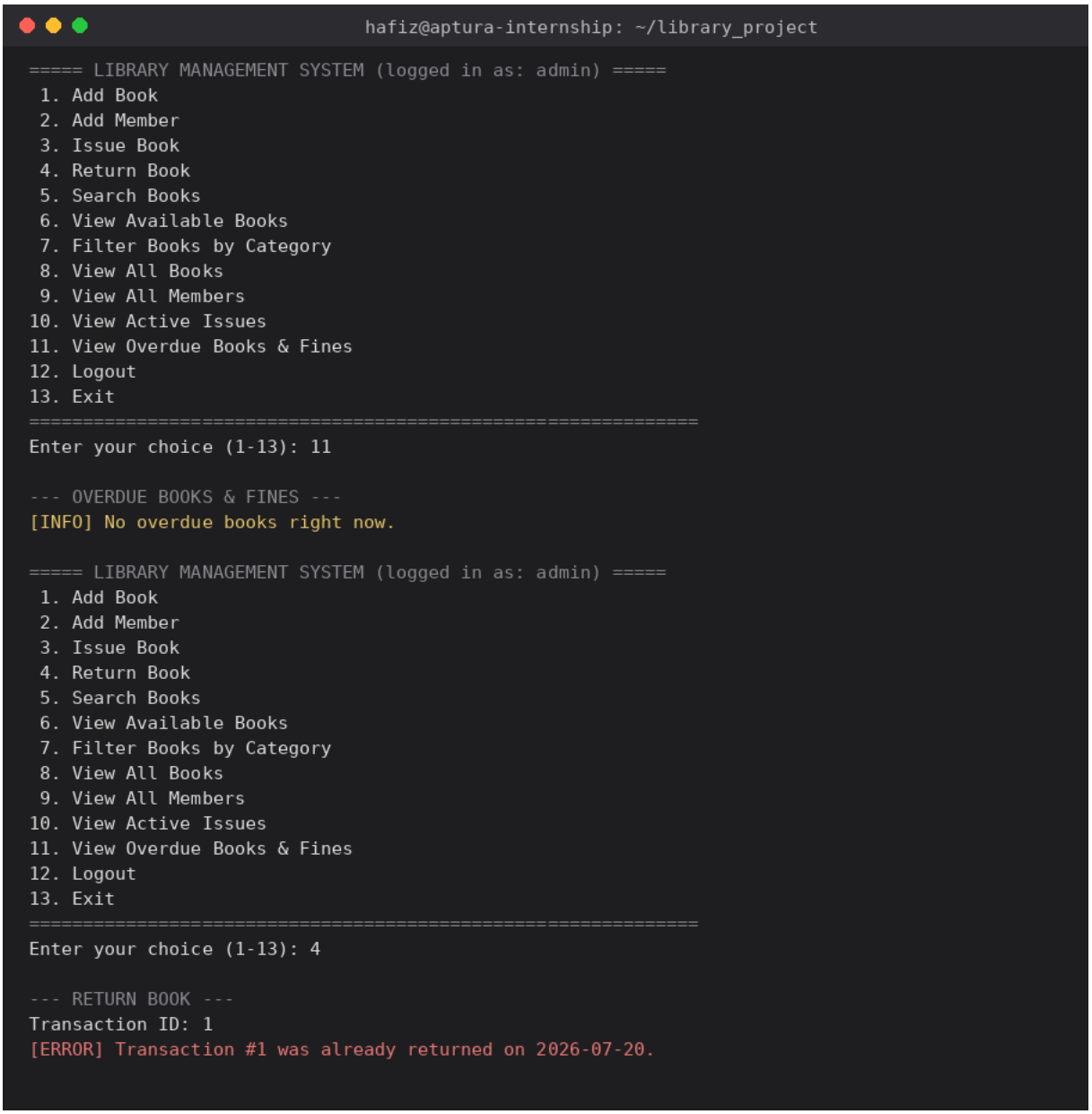
Viewing the full book catalogue.



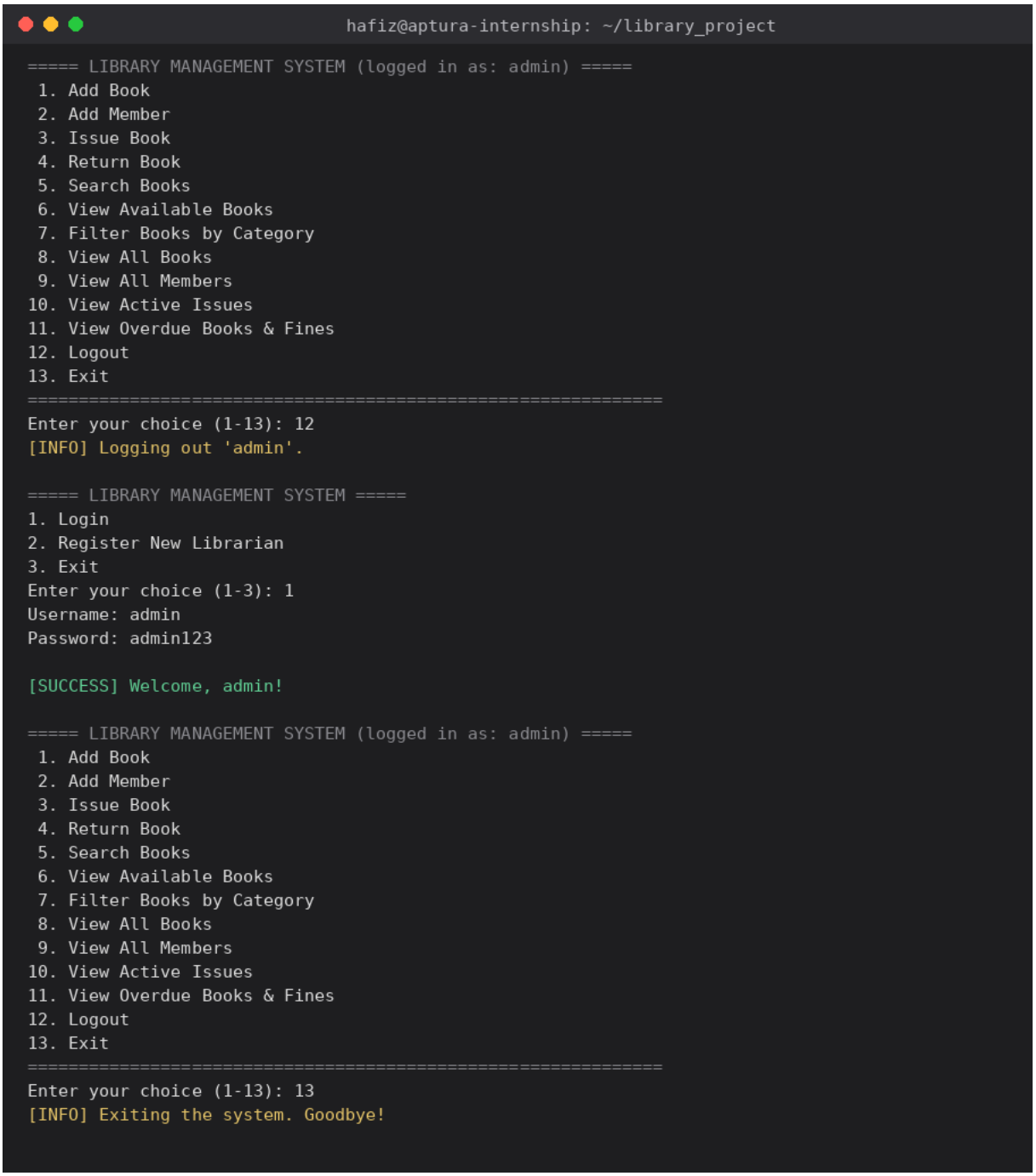
Viewing all members and the currently active (unreturned) issues.



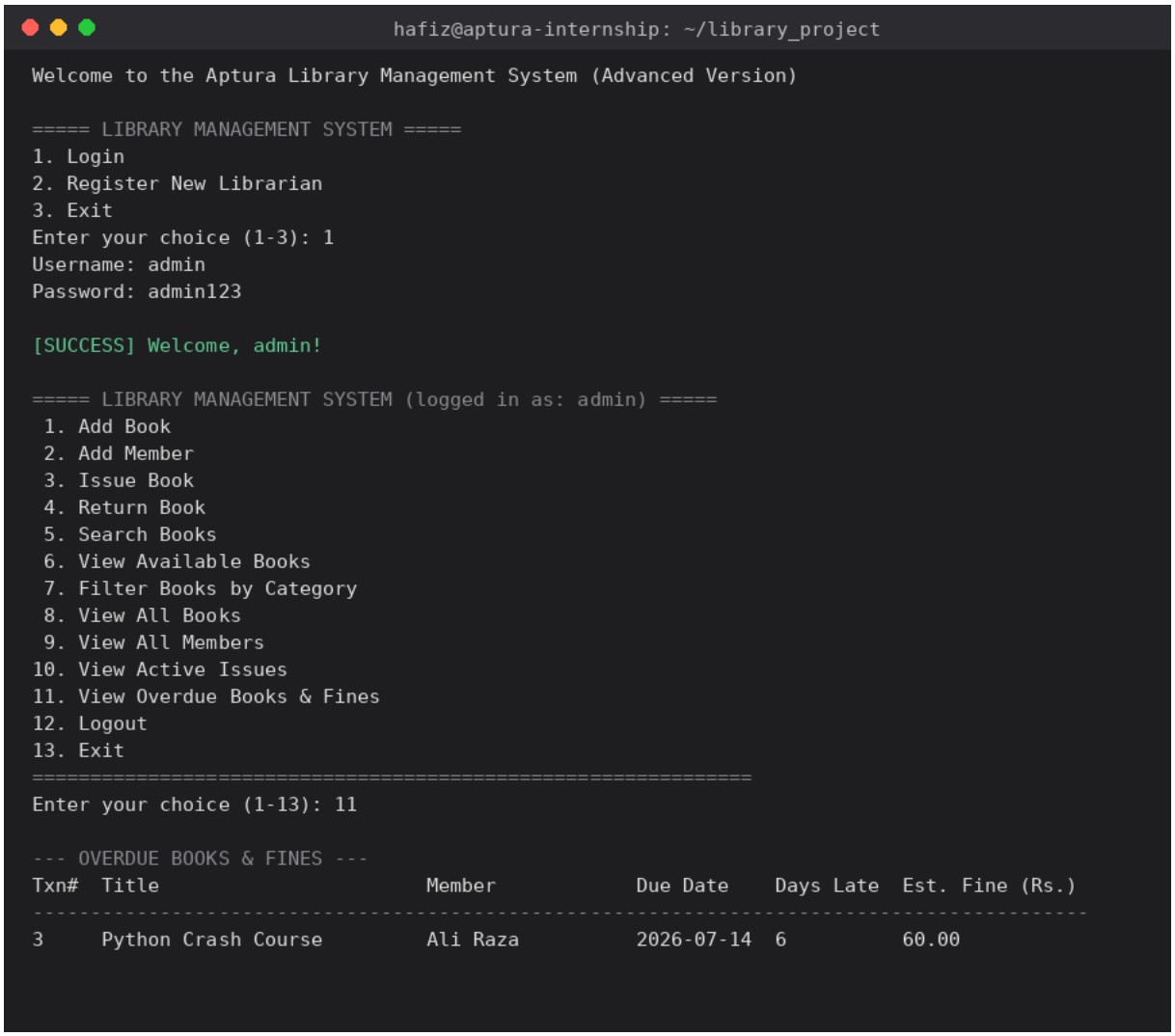
Returning a book on time – no fine charged.



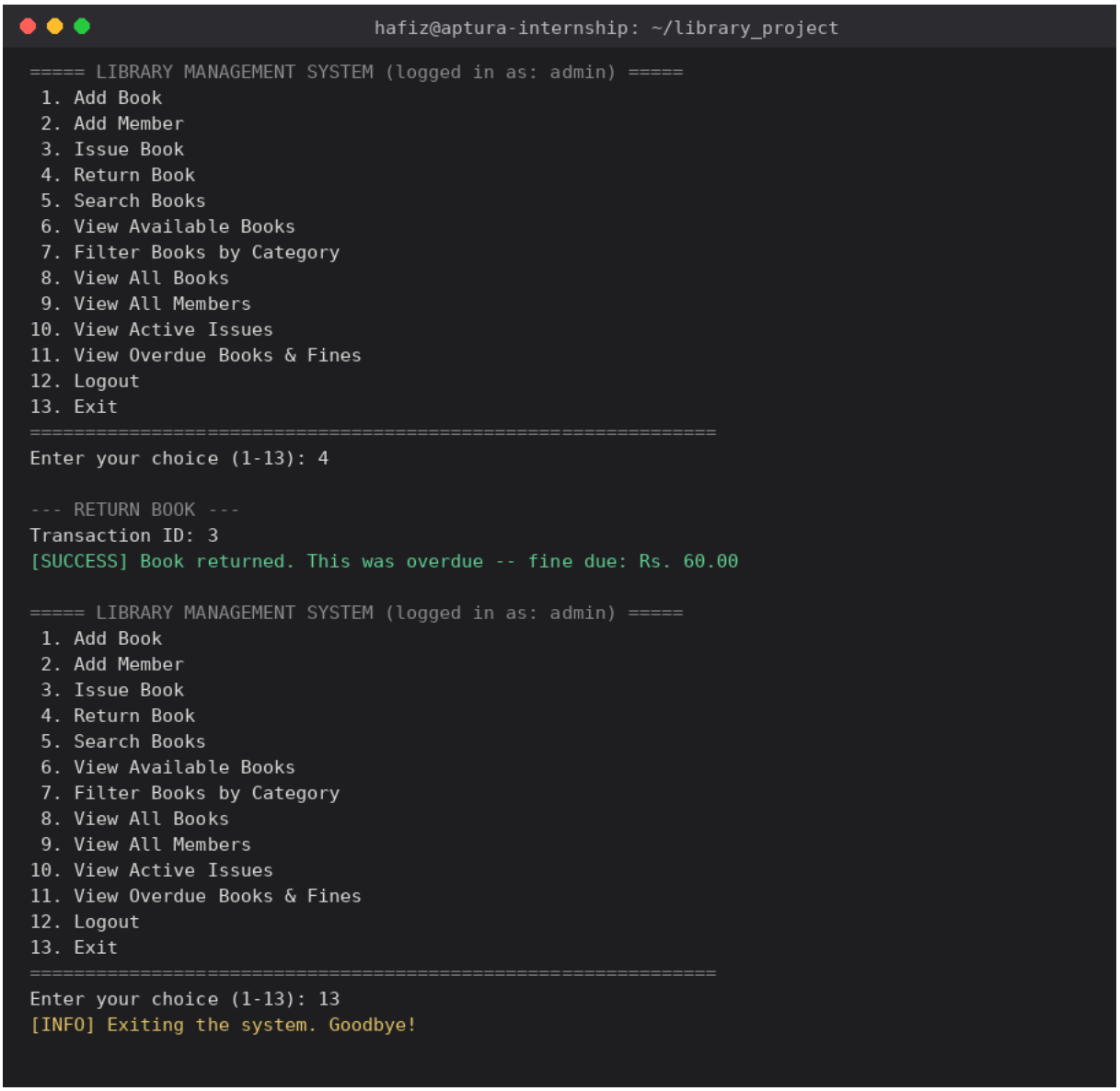
Overdue report (empty, since nothing is due yet) and a rejected attempt to return the same book twice.



Logging out, logging back in, and exiting cleanly.



Dedicated fine-calculation demo (Section 4.1 evidence continued): a transaction seeded as 6 days overdue is correctly detected and its fine estimated at Rs. 60.00.



Returning that same overdue transaction through the real production return_book() code path charges the exact same fine: Rs. 60.00.

7. Conclusion

This task combined the week's core skills – object-oriented design, file/database handling, and structured exception handling – into a single system, then pushed further into two system-design questions grounded in the actual implementation rather than answered in the abstract: the concurrency protection in Question 01 was empirically stress-tested, and the storage choice in Question 02 was the same engine actually used in the code. All deliverables – source code, this report, execution screenshots, and the project flow diagram – are submitted together for Week 02 of the Python Development track.